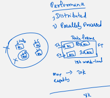
ML-Scale

Performance
1) Distributed
2) Parallely Processed



max capacity → 20k

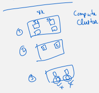
use-case → 8k
capacity = 5k
N1 latency → High → performance degradatⁿ
N2

Optimizatⁿ

Resources → Idle

Cost

Not effective



Compute Cluster ⇒ All purpose Cluster (Clean) → Exploratⁿ & Research → Not effective
↳ Job cluster → Fixed size → development Env. → 50%
Serverless cluster → Auto scalable → Deployment (prodⁿ) → 80%
SQL warehouse → Analytics

Dataframes → Distribution of W.L.
Partitions → Searching → skip unseen partition
→ partitⁿ where data exists

} Liquid Clustering (Delta Lake)
→ Vacuum
→ Deletion vector
→ Partition → Cluster By (ANN, CNN)
→ Time travel → versioning

Python + Apache Spark ⇒ PySpark
↓ Dataframes (Distribution)
↘ Pandas (Visualitⁿ) API

Delta Lake & Unity Catalog
↓
Governance & Storage

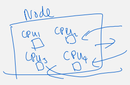
JVM → Threading
↓
tasks → T1, T2, T3 → Optuna

Node1
Threads
T1, T2, T3

Distributed

Dataset
N1 N2 N3 → Ray tune

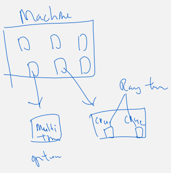
Hyperparameter → Optuna.



→ Ray tune

Latency → low → HPO

Hyperparameter (Performance & optimatⁿ) → Clusters (Apache Spark) →
↙ Optuna
↘ Ray tune



Pre-seam ← Align Model
↓
Features Store
↓
prediction

Low Latency

Performance & optimatⁿ evalⁿ

Deployment

Model
↙ 0.8 Train Ver1
↘ 0.2 test Ver2

Distribute work load → Serving Endpoint (DB's feature)
+
Inference (Testing)

Development → Staging (ML Ops) → Deployment
+
A. spark
[DF, Pandas API]

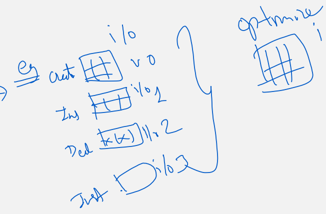
Staging
↓
CI/CD
DAB

Deployment
+
A. spark
(DF, Pandas & API)
↓
Model registry + versions
↓
alias

Lab 3.2 Performance & optimizatⁿ → Delta Lake

} Liquid Clustering (Delta Lake)
→ Vacuum
→ Deletion vector
→ Partition → Cluster By (ANN, CNN)
→ Time travel → versioning
→ Optimize

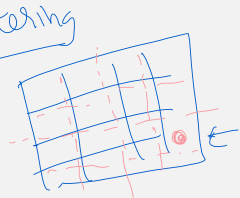
1) optimize → Solve - issues of small files
↓
Creating a Single file



2) Vacuum → Del. Retention Period → 7 days

5
6
7
8th
X → Clean

3) Clustering



← specify ANN / KNN

skipping → save time

Partitioning → vector search (Index)
(Unity Catalog)